

Processamentos de imagens – resumo prova II

Funções de transformação de histograma:

Link dos slides melhores: <http://www.facom.ufu.br/~backes/gsi058/>

Histograma:

O histograma de uma imagem é um conjunto de números que indica a quantidade de pixels em cada um dos níveis de cinza da imagem.

Para o que são usados as funções de transformação de histograma? -

Alterar a distribuição de probabilidade dos tons cinzas e aumentar a percepção visual (aumentando basicamente o contraste).

Características Importantes:

- As informações espaciais não são representadas;
- Um histograma é único para uma determinada imagem, mas o inverso não é verdadeiro;
- A movimentação de objetos em uma imagem não tem qualquer efeito sobre o seu histograma.

Equalização

Definição

Técnica onde se procura redistribuir os valores dos níveis de cinza em uma imagem, para se obter um histograma uniforme.

Visa aumentar o intervalo dinâmico de uma imagem melhorando o contraste de imagens adquiridas sob péssimas condições de iluminação

Vantagem

- Por utilizar esse critério da frequência de tons de cinzas, este não necessita de argumentos/parâmetros além da própria imagem (De modo geral o valor máximo é sempre colocado como o valor máximo dos tons que é 255).

Desvantagem

- A equalização só funciona bem, quando o objeto de interesse ocupa a maior parte da imagem, se isso acontecer, não é bom utilizar a equalização pois o resultado será um fundo com mais contraste do que o próprio objeto.
 - Solução: Usar janelamento, pois você poderá especificar a faixa de tons de cinza a ser expandido

Como calcular:

Nomenclaturas:

s = valor do pixel

h(s) = frequência do pixel “s”

somatório(s) = somatório de h(0) até h(s)

r = novo valor do pixel s, que pode ser calculado assim $\rightarrow \text{round}(\text{Somatório}(s)/n * \text{MAX})$

MAX na maior parte dos casos sempre será 255 que é o maior valor que um tom de cinza pode ter.

Exemplo

Considere a imagem abaixo

2	1	0	0	0
2	22	23	22	0
2	22	24	1	1
0	24	23	1	0
0	2	0	0	0

Etapas

1º passo – fazer histograma dos tons de cinza

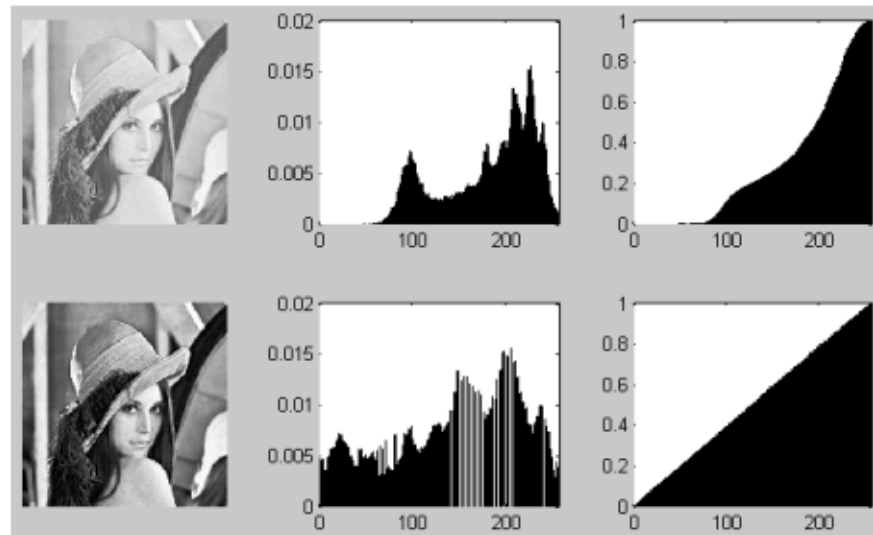
s	h(s)	Somatório h	r
0	10	10	102
1	4	14	143
2	4	18	184
22	3	21	214
23	2	23	235
24	2	25	255

2º passo - substitua os valores de R de cada um dos tons para a imagem

184	143	102	102	102
184	214	235	214	102
184	214	255	143	143
102	255	235	143	102
102	184	102	102	102

Exemplos gráficos

Equalização do Histograma



Filtragem espacial

Convolução

Processo onde uma kernel é passada por cima de uma imagem para aplicar um filtro

Exemplos:

- Passa Baixa e média espacial (suavização)
- Filtragem mediana Passa Alta (realce)
- Passa banda
- Gradientes (robert, sobel, etc): detectores de borda

Multiplicar os valores da kernel em cada pixel de uma imagem,

Obs: a máscara de convolução de kernel precisa ser espelhada antes de fazer a operação

Exemplo de uma convolução com uma imagem qualquer

5	5	5	9	9	9
5	5	5	9	9	9
5	5	5	9	9	9
5	5	5	9	9	9
5	5	5	9	9	9
5	5	5	9	9	9

Essa é a kernel:

1	1	1
1	1	1
1	1	1

Operação, multiplicação ponto a ponto e depois somar tudo, depois(se especificado), dividir pelo tamanho da kernel

Observe o exemplo

5	5	5	9	9	9
5	5	5	9	9	9
5	5	5	9	9	9
5	5	5	9	9	9
5	5	5	9	9	9
5	5	5	9	9	9

$$(5 * 1 * 3) + (6 * 1 * 9) / 9$$

$$15 + 36/9$$

51/9 → dividir e arredondar para baixo

Esse resultado, vai ficar no centro da matriz da kernel, em uma nova matriz (para não atrapalhar os cálculos)

A imagem resultado é menor do que a imagem original. Os valores marcados com * não podem ser calculados

*	*	*	*	*	*
*			5		*
*					*
*					*
*					*
*					*
*	*	*	*	*	*

Preço

O custo computacional da convolução é alto. Em uma imagem de tamanho $M \times M$ e máscara $N \times N$, o número de multiplicações é de $M^2 \times N^2$. Exemplo: imagem de 512×512 e máscara de $16 \times 16 = 67.108.864$ multiplicações.

Mediana

Como fazer: Considerando uma kernel 3×3 , achar no espaço pertencente à imagem, o pixel que contém o valor que está no centro da distribuição.

Considere a imagem a seguir:

3	5	2	1	1
1	4	6	2	1
1	1	5	6	2
1	1	1	1	1
1	2	2	2	1

1º Passo - colocar a kernel

3	5	2	1	1
1	4	6	2	1
1	1	5	6	2
1	1	1	1	1

1	2	2	2	1
---	---	---	---	---

2º Passo - Achar a mediana da distribuição:

```
numbers = [3,5,2,1,4,6,1,1,5]
numbers.sort()
print(numbers)
[1, 1, 1, 2, 3, 4, 5, 5, 6]
```

Perceba que a mediana neste caso é o 3, pois está no centro da lista

3º Passo - Colocar o valor em uma nova matrix

	3			

4º Passo - Repetir passos anteriores

Vantagem:

- remoção de ruídos sem perdas visíveis de bordas.

Desvantagem:

- Detalhes que são muito pequenos podem ser perdidos.
- O ruído só será removido, se este for menor do que o da máscara

Média:

Faz as médias dos pixels para realizar uma suavização dos pixels, reduzindo o ruído.

Como fazer: Considerando uma kernel 3x3, somar todos os pixels então dividir pela quantidade de pixels na kernel (em uma kernel 3x3, iríamos dividir por 9)

Exemplo:

Considere a imagem a seguir:

3	5	2	1	1
1	4	6	2	1
1	1	5	6	2
1	1	1	1	1
1	2	2	2	1

1º Passo - colocar a kernel

3	5	2	1	1
1	4	6	2	1
1	1	5	6	2

1	1	1	1	1
1	2	2	2	1

2º Passo - Achar a média da distribuição:

média = $\text{round}((3 + 5 + 2 + 1 + 4 + 6 + 1 + 1 + 5)/\text{tamanho_kernel})$

média = 3

3º Passo - Colocar o valor em uma nova matrix

	3			

4º Passo - Repetir passos anteriores

Sobel

Detectar bordas de objetos em uma imagem, a partir do encontro de áreas com frequência alta:

Exemplo

Considere a imagem a seguir

2	1	0	0	0
2	22	23	22	0
2	22	24	1	1
0	24	23	1	0

0	2	0	0	0
---	---	---	---	---

Etapas

1- Multiplicar a imagem com a kernel abaixo, gerando $G(x)$ ou $S(x)$:

-1	0	1
-2	0	2
-1	0	1

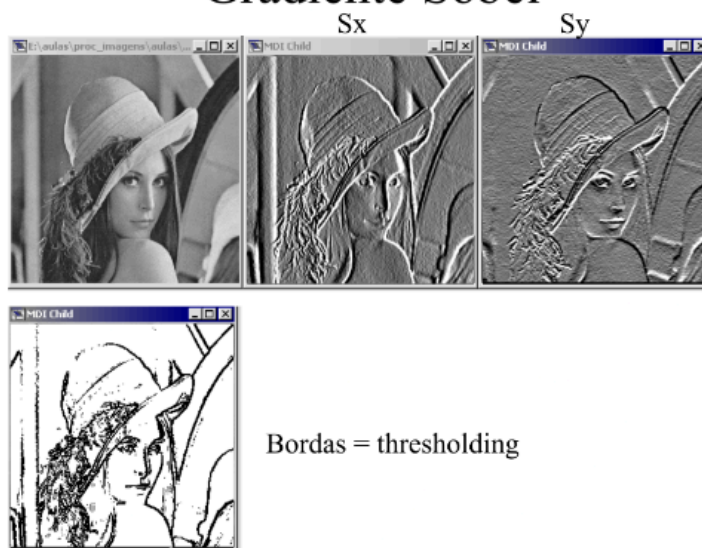
2 - Multiplicar a imagem com a kernel abaixo, gerando $G(y)$ ou $S(y)$:

-1	-2	-1
0	0	0
1	2	1

3 - Achar o módulo das etapas 1 e 2 e então somá-las

$$S = |S(x)| + |S(y)|$$

Gradiente Sobel



Detector de borda (primeira derivada)

Canny

Detectar bordas contínuas

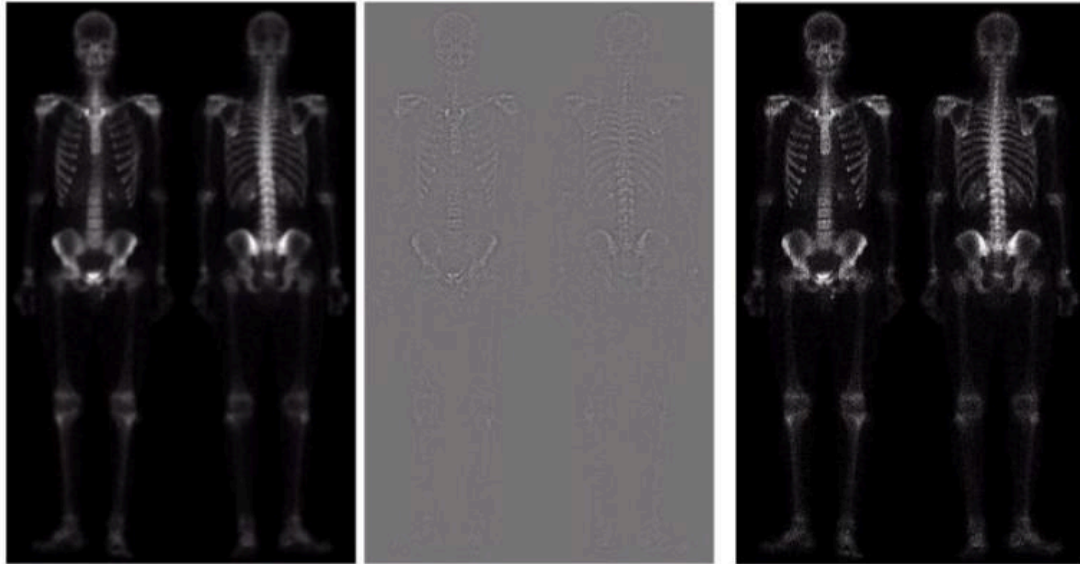
tentar limpar as imagens corrompidas por ruído branco (convolução gaussiana)

Etapas

1. – Suavizar uma imagem, usando convolução gaussiana (por causa dos ruídos)
 - $F_s(x,y) = f(x,y) * G(x,y)$
2. Para cada pixel, computar a magnitude $M(x,y)$ e a direção do gradiente $\alpha(x,y)$
 - Usar qualquer filtro de primeira derivada (Sobel)
 - $M(x,y)$ fatalmente conterà cristas largas em torno dos máximos locais. Para afinar estas cristas utilize algoritmo de supressão dos não máximos, afinando $M(x,y)$
3. Limiarizar $M(x,y)$ e analisar a conectividade → (partes das bordas podem conter falhas)

Filtro de Laplace/Laplaciano(Sharpen)

Faz o realce das bordas de uma imagem, deixando elas mais nítidas. Basicamente, será criada uma imagem onde as bordas estão realçadas, e então esta será somada com a imagem original, gerando em uma imagem mais nítida.



Desvantagem

Se a imagem tiver ruído, este também será realçado

Transformada de Fourier

Transformar o domínio de imagem(coordenadas da imagem) para um domínio de frequência(informações sobre variações lentas ou rápidas no NC em uma imagem)(multiplicação ponto-a-ponto)

Domínio de imagem $\rightarrow$ DFT $\rightarrow$ Domínio da frequência

DFT $O(p \cdot gp)$

Domínio da frequência $\rightarrow$ IDFT $\rightarrow$ Domínio da Imagem

IDFT $O(p \cdot gp)$

No domínio de imagem

$O(p^2)$

No domínio da frequência

$2 O(p \cdot gp) + O(p) = O(p \cdot gp)$

Frequências imagem (variação dos tons de cinza em uma imagem)

Baixa frequências → regiões homogêneas da imagem (sem muita variação nos tons de cinza)

Alta frequência → regiões onde há grande oscilação dos tons de cinza (textura, bordas)

Obs: o que determina uma região de alta ou baixa frequência é a quantidade de tons de cinza da imagem

Domínio da imagem: x, y e tom

Domínio da frequência: freq u, v, amplitudes

Sinal 1D:

$F(u) = \int_{-\infty}^{\infty} f(x) \exp[-j2\pi u x] dx$

$f(x) \rightarrow$ pixel

$j \rightarrow$ número complexo = $(-1)^{1/2}$

$u \rightarrow$ frequência que estou calculando

$\exp \rightarrow$ exponencial

Por estar expressa no domínio complexo, a Transformada de Fourier possui componente real e imaginário.

Sinal 2D

$F(u,v) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x,y) \exp[-j2\pi(ux + vy)] dx dy$

Imagem:

Original > eixo de coordenadas(x,y) de 0 até tamanho da imagem

Transformada de Fourier > eixo de coordenadas(b,v) de 0 até N

TF deslocado > de 0 até $n=N/2$

Transformada Discreta de Fourier

Seja $f(x)$ uma função contínua, podemos discretizar esta função escrevendo-a da forma

$f(x) = f(x_0 + x * \Delta X)$

$F(u) = \frac{1}{n} * \text{Somatório de } x=0 \text{ até } n-1 f(x) \exp[-j2\pi u * x/N]$

$F(u,v) = \frac{1}{mn} * \text{Somatório de } M-1 \text{ e } N-1 \exp[-j2\pi (ux/M + vy/N)]$

Construindo uma DFT

Ex. construa a DFT da função abaixo.

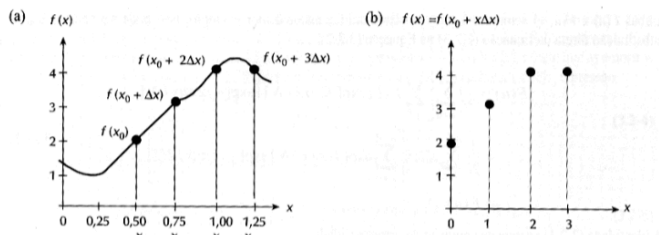


Figura 3.5 – Uma função simples e amostras no domínio de x . Em (a) x é uma variável contínua; em (b) x é discreta.

$N =$

$F(0) =$

$\Delta x =$

$F(1) =$

$F(2) =$

$F(3) =$

Calcular também o espectro $|F(u)|$ de DFT e plotar o gráfico.

$N = 4$

$F(x): 0 \leq x < 4 \rightarrow$ Imagem com 4 pixels

2	3	4	4
---	---	---	---

Obs.: $F(U) = au + i * bu$

a	-	-	-	-
b	-	-	-	-
u	0	1	2	3
	-4	-3	-2	-1

Obs.: $f(x) \rightarrow$ tom de cinza

$F(U) = 1/n * (\text{Somatório } x = 0 \text{ até } n-1 [f(x) \cos ((2\pi u x)/n) - (i \text{ sen } ((2\pi u x)/n))])$

$u \rightarrow$ frequência

$n \rightarrow$ número de pixels

$x \rightarrow$ coord do pixel

$F(0) = 1/4 * \text{Somatório de } 0 \text{ até } 3 \text{ sendo } 3 = N-1$

$$\text{Somatório } x = 0 \rightarrow [2 (\cos((2*\pi*0)/4) - i \sin 0)]$$

$$\text{Somatório } x = 1 \rightarrow [3 (\cos((2*\pi*1)/4) - i \sin 0)]$$

$$\text{Somatório } x = 2 \rightarrow [4 (\cos((2*\pi*2)/4) - i \sin 0)]$$

$$\text{Somatório } x = 3 \rightarrow [4(\cos(0) - i \sin 0)]$$

$$\cos(0) \rightarrow 1$$

$$\sin(0) \rightarrow 0$$

Conta final

$$\frac{1}{4} * [2 + 3 + 4 + 4] = 13/4 \rightarrow \text{representa a média da imagem}$$

$$13/4 = a = \text{parte real}$$

$$0 = b = \text{parte imaginária}$$

Colocando esse dado na tabela

a	13/4	-	-	-
b	0	-	-	-
u	0	1	2	3
	-4	-3	-2	-1

Lembrando que a parte do Cos é a parte do (A) da função

e o seno é a parte do (B) da função (parte imaginária)

Complexidade : $O(n^2)$

Obs.: pode ser reduzido a complexidade utilizando programação dinâmica.

$$F(1) = \frac{1}{4} * \text{Somatório de } x \text{ que vai de } 0 \text{ até } 3 \text{ sendo } 3 = N-1$$

$$\text{Somatório } x = 0 \rightarrow [2 (\cos((2*\pi*1*0)/4) - i \sin 0)]$$

$$\text{Somatório } x = 1 \rightarrow [3 (\cos((2*\pi*1*1)/4) - i \sin \pi/2)]$$

$$\text{Somatório } x = 2 \rightarrow [4 (\cos((2*\pi*1*2)/4) - i \sin \pi)]$$

$$\text{Somatório } x = 3 \rightarrow [4(\cos((2*\pi*1*3)/4) - i \sin 3\pi/2)]$$

Conta final

$$\frac{1}{4} * [2 + 3[0-i] + 4[-1] + 4[i]] = -1/2 + i/4 \rightarrow \text{representa a média da imagem}$$

$$-1/2 = a = \text{parte real}$$

$1/4 = b$ = parte imaginária

Colocando esse dado na tabela

a	13/4	-1/2	-	-
b	0	1/4	-	-
u	0	1	2	3
	-4	-3	-2	-1

Resultado final

$F(U) =$

a	13/4	-1/2	-1/4	-1/2
b	0	1/4	-1/4	-1/4
u	0	1	2	3
	-4	-3	-2	-1

A média do “u” igual a 1 é bem parecido com o “u” igual a 3, com apenas a parte imaginária com sinal inverso. Ambos os números são conjugados.

Transformada rápida de Fourier, explora as particularidades das contas, para evitar cálculos repetidos, conseguindo complexidade $O(n \cdot \log(n))$

Propriedades da transformada de Fourier:

Translação

Em vez de refazer os cálculos de Fourier durante uma translação, é só multiplicar pela constante $\exp(-i2\pi u/N)$ cada valor em $F(u)$.

Rotação

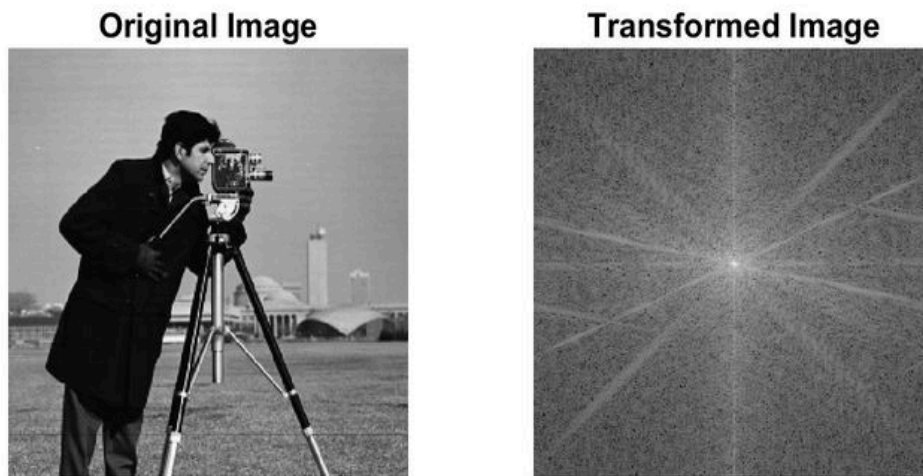
Em vez de refazer os cálculos de Fourier durante uma rotação, é só multiplicar pelo **ângulo da rotação** cada valor em $F(u)$.

Essas operações estão na ordem de $O(n)$, enquanto que refazer o Fourier seria no mínimo $O(n \cdot \log(n))$ e no máximo $O(n^2)$,

Periodicidade

$$F(u) = F(u+N)$$

Em outras palavras, o espectro de fourier se repete em todos os lados



Simetria conjugada

$$F(u) = F^*(-u)$$

Ou seja

a	a0	a1	a2	a3	a4=a2	a5=a1
b	b0	b1	b2	b3	b4=-b2	b5=-b1
u	0	1	2	3	4	5
				-3	-2	-1

Isso significa que podemos apenas calcular metade da matriz de Fourier e a outra metade apenas colocar na forma repetir os valores e colocar a parte imaginária na forma conjugada, diminuindo os cálculos pela metade.

Transformada Discreta Inversa de Fourier

Usando como entrada uma matriz de frequência, conseguimos pegar de volta a imagem que pertence a frequência.

fórmula:

$$f(x) = (\text{Somatório de } 0 \text{ até } n-1 [F(u) (\cos((2\pi * u * x)/n) - i \sin((2\pi * u * x)/n))]$$

Convolução discreta

Teorema de convolução (mais usadas com uma máscara grande)

Fazer a transformada com $f(t) * g(t)$ é igual a multiplicar a transformada de $F(u)$ com a transformada de $G(U)$

E multiplicar do domínio de $f(t) * g(t)$ é igual a fazer transformada inversa de $F(u)$ com $G(u)$ isso substitui uma operação que antes era $O(n^2)$ por uma que é $O(n \cdot \log(n))$

Como utilizar Transformada de Fourier

$g(x,y) = \text{Fourier inverso} (H(u,v) * F(u,v))$

$g(x,y)$ é a nova imagem processada

$H(u,v)$ é chamada função de transferência (filtro)

$F(u,v)$ é a transformada de Fourier de $f(x,y)$

$f(x,y)$ é a imagem original.

Obs.: a máscara($H(x,y)$) e a imagem tem que ser do mesmo tamanho, caso elas tenham tamanhos diferentes

a máscara será expandida artificialmente.

Exemplos de Filtro de Fourier

Filtro Ideal Passa-Baixa

Ele vai “Limiarizar” o $F(u,v)$ de Fourier, preservando as frequências do meio do círculo.

Exemplo

considerando $F(u) =$

a	1.2	0.06	0.1	0.1	0.06
b	0	0.04	-0.5	0.5	-0.04
u	0	1	2	3	4

	-5	-4	-3	-2	-1
--	----	----	----	----	----

Passando filtro de passa-baixo $|u| < 2$

a	1.2	0.06	0	0	0.06
b	0	0.04	0	0	-0.04
u	0	1	2	3	4
			-3	-2	-1

Os lugares em **vermelho** foram multiplicados por 0.

Filtro ideal Passa-Alta

contrário da Passa-baixa, ele vai preservar as frequências fora do círculo

EF: 2 Imagens: a real e imaginária

EP: 1 Imagem

Filtragem especial: escolhe frequências específicas para remoção de ruído

Aplica a transformada de fourier, faz um plot dessa imagem, aplica uma escala logaritmica (fazendo os valores pequenos ficar mais ressaltados), analisar os pontos “inaturais” da imagem (no olho), zerar esses pontos na matriz . Por fim, realizar a transformada inversa de Fourier para obter a imagem.

Esse processo é difícil para ser realizado por máquina, necessitando ser treinada a partir de algum tipo de classificador, ou se não, fazer uma equalização

Classificar uma frequência alta e baixa é bem subjetiva, pois depende do problema

NC → Níveis de cinza

JPEG Standard – Um dos padrões de compressão mais importantes que temos

JPEG foi feito onde a perda de dados é suportada (formato sem perda possui compressão pior que outros métodos)

Por esse motivo ele não é usado na área médica.

Mais usado hoje: Sequencial → Baseline

Compressão é um compromisso em três coisas: complexidade do algoritmo, a qualidade da compressão e qualidade da imagem.

O projeto principal se caracteriza na visão humana, sacrificar as altas frequências e número de tons de cinzas, algo que os seres humanos possuem dificuldade de enxergar. Esse é o motivo da alta compressão, mantendo uma qualidade que ainda é aceitável.

Passo-a-passo

Pegar a base RGB da imagem e passa para o modelo YCbCr (crominância e luminância), passar blocos de 8x8 pela transformada discreta de cossenos, depois estes números serão quantizados (cortar as casas decimais a partir de uma tabela)(arredondamento) gerando número menores, que são mais fáceis de comprimir.

Depois disso, a compressão se divide em duas timelines: DC (frequência zero) frequências, e o AC(Alta frequência).

Conversão de cores

Perda de quantização (downsampling) (compressão)

4:4:4 retenção

4:2:2 diminuir a crominância pela metade

4:2:0 diminuir a crominância em $\frac{1}{4}$

Transformada discreta de cossenos:

Dada a função $f(x) \cos \omega x$, definida no intervalo $0 \leq x < 2$

$\omega = 2\pi / P = \pi$

onde P é o período, ou, intervalo

$$\cos \pi = -1$$